The Theory of Computing

Context Free Grammars

The rules governing well-formed mathematical expressions are an example of a context free grammar.

$$(3 * 6 + 5) * \left(8 - \frac{12}{3}\right)$$

Recall your order of operations.

$$(3 * 6 + 5) * \left(8 - \frac{12}{3}\right)$$

Recall your order of operations.

$$(18 + 5) * (8 - 4)$$

Recall your order of operations.

$$(18 + 5) * (8 - 4)$$

Recall your order of operations.

$$23 * 4$$

Recall your order of operations.

$$23 * 4$$

Recall your order of operations.

92

Recall your order of operations.

$$(3 * 6 + 5) * \left(8 - \frac{12}{3}\right)$$

How does a compiler know if a mathematical or logical expression is well-formed?

$$4 + 6 * 5 - 4 * 2$$
$$x > 12$$
$$x \wedge (y \vee z)$$

Well-formed mathematical and logical expressions

$$4 +* 5$$
$$> 100$$
$$(x + y$$

Ill-formed mathematical and logical expressions

Let’s restrict ourselves to a subset of expressions using only numbers, addition, multiplication and parentheses.

Definition: An expression E is a well-formed mathematical expression if E is:

a. a string of digits or bits (i.e., a number),

b. (E_1) where E_1 is a well-formed mathematical expression,

c. $E_1 + E_2$ where E_1 and E_2 are well formed mathematical expressions, or

d. $E_1 * E_2$ where E_1 and E_2 are well formed mathematical expressions.

We can formalize this recursive structure in a set of substitution rules called a context free grammar:

$$E \rightarrow (E) \mid E + E \mid E * E \mid N$$
$$N \rightarrow NN \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

A similar grammar works for the binary alphabet:

$$E \to (E) \mid E + E \mid E * E \mid N$$
$$N \to NN \mid 0 \mid 1$$

$$L = \{0^n 1^n \mid n \geq 0\}$$

Recall this language which we have shown is not regular. Can we create a context free grammar for L?

$$L = \{0^n 1^n \mid n \geq 0\}$$

$$S \rightarrow 0S1 \mid \epsilon$$

Definition: A derivation of a string w is a sequence of substitutions of a context free grammar G that yields the string w.

$$S \rightarrow 0S1 \mid \epsilon$$

We can see that S derives ϵ.

$$S \Rightarrow \epsilon$$

$$S \rightarrow 0S1 \mid \epsilon$$

We can see that S derives 0011

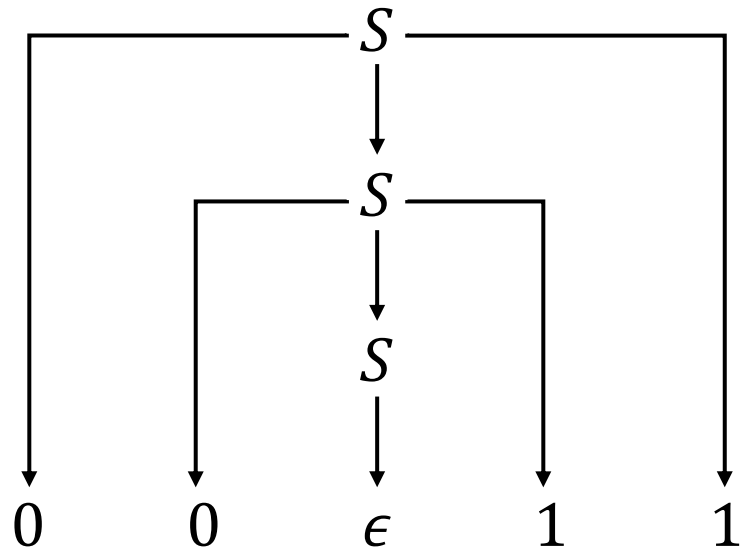
$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$$

$$E \to (E) \mid E + E \mid E * E \mid N$$
$$N \to NN \mid 0 \mid 1$$

We can see that E derives $10 + (11 * 10)$

$$E \Rightarrow E + E \Rightarrow N + (E) \Rightarrow NN + (E * E) \Rightarrow$$
$$10 + (N * N) \Rightarrow 10 + (NN * NN) \Rightarrow$$
$$10 + (11 * 10)$$

Derivations can be represented with a parse tree:



A context free grammar generates strings in a language by following all possible substitution paths.

Definition: A context free grammar (CFG) is a 4-tuple (V, Σ, R, S), where:

1. V is a finite set of variables,
2. Σ is a finite set of terminals (Σ must be disjoint from V),
3. R is a set of rules, with each rule mapping a variable to a string of variables and terminals,
4. $S \in V$ is the start variable.

Definition: If u, v and w are strings of variables and terminals and $A \to w$ is a rule we say that uAv yields uwv and we write this as $uAv \Rightarrow uwv$.

Definition: We say that u derives v, written $u \overset{*}{\Rightarrow} v$, if $u = v$ or if a sequence $u_1, u_2, \ldots, u_k$ exists for $k \geq 0$ and $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v$.

Definition: The language of a grammar G is $L(G) = \left\{ w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w \right\}$.

Exercises

$L_0 = \Sigma^*$

$L_1 = \{w | w \text{ has at least three } 1s\}$

$L_2 = \{w | \text{ the length of } w \text{ is odd}\}$

$L_3 = \{w | w \text{ starts and ends with } 00\}$

$L_4 = \{w | w \text{ is a palindrome}\}$

$L_5 = \{0^n 1^m 0^m 1^n | n, m \geq 0\}$

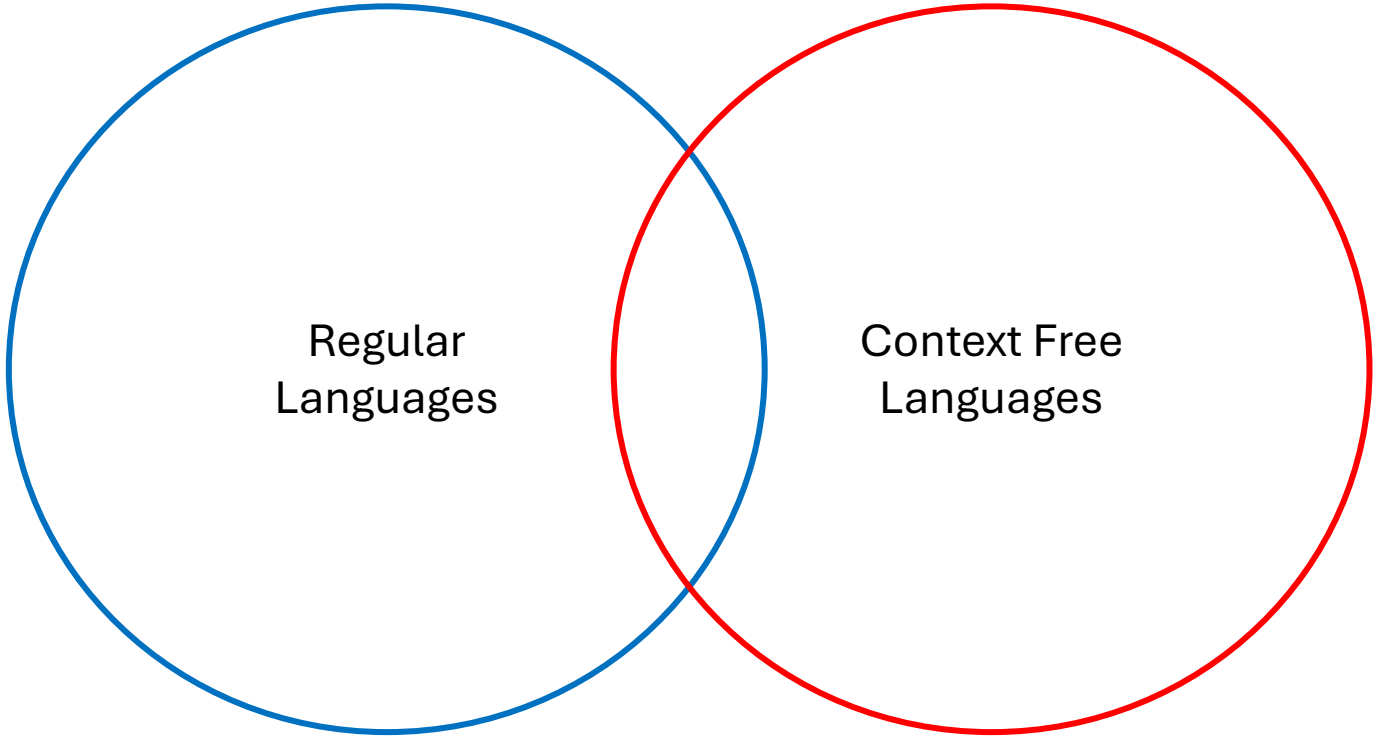
$L_6 = \{w | \text{ the middle symbol of } w \text{ is a } 0\}$

$L_7 = \{w | \ w \text{ contains equal numbers of } 0s \text{ and } 1s\}$

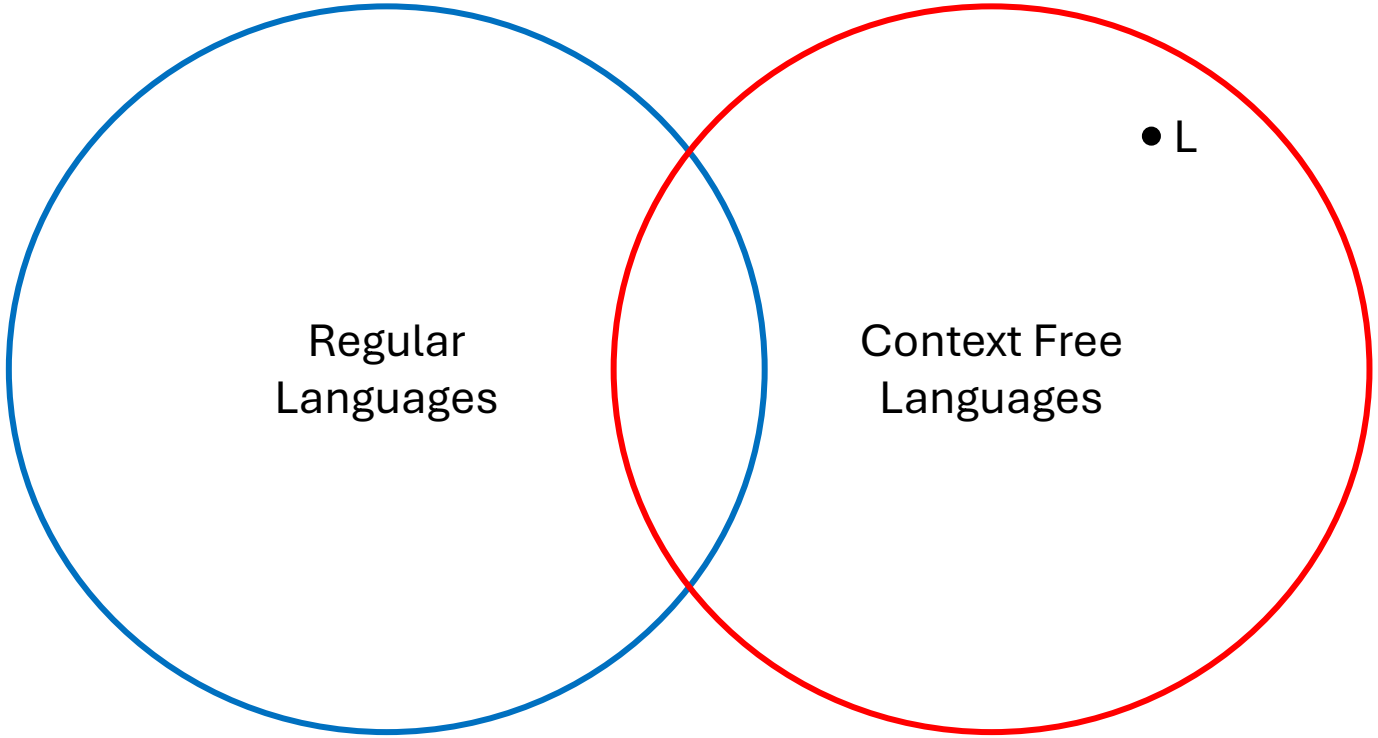
The Theory of Computing

Context Free Languages

Definition: A language L is a context free language (CFL) if there is a context free grammar G such that $L(G) = L$.

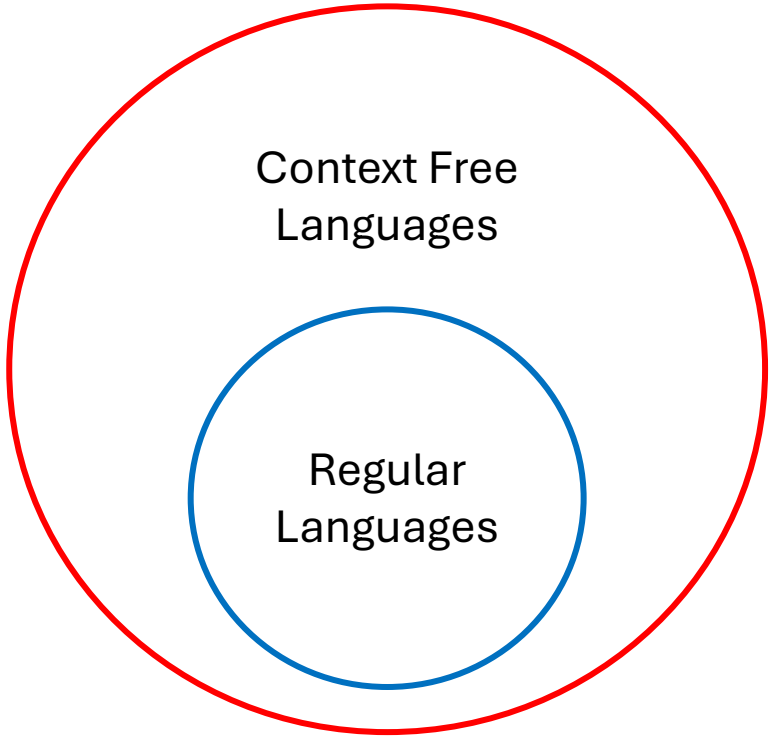


What is the relationship between the regular languages and the context free languages?



We have shown already one language that is context free, but not regular.

$$L = \{0^n 1^n \mid n \geq 0\}$$



Theorem: If a language is a regular language, then it is a context free language.

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

Theorem: If a language is a regular language, then it is a context free language.

Proof: Assume L is a regular language and let R be a regular expression such that $L(R) = L$. By the definition of a regular expression, we have six possible cases we must handle.

1. If $R = \emptyset$ we can create the grammar $S \to \emptyset$.
2. If $R = \epsilon$ we can create the grammar $S \to \epsilon$.
3. If $R = \sigma \in \Sigma$ we can create the grammar $S \to \sigma$.
4. If $R = R_1 \cup R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 | S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
5. If $R = R_1 \cdot R_2$ we can create a new grammar out of grammars for R_1 and R_2. We create a new start rule $S \to S_1 S_2$ where S_1 and S_2 are the start rules of R_1 and R_2.
6. If $R = R_1^*$ we can create a new grammar out of a grammar for R_1. We create a new start rule $S \to SS | S_1 | \epsilon$ where S_1 is the start rule of R_1. □

The Theory of Computing

Ambiguity in Context Free Grammars

“The girl touches the boy with the flower.”

Who has the flower?

This sentence has two correct grammatical interpretations and thus is ambiguous.

$$a + b * c$$

Without order of operations this is also ambiguous.

$$G\colon\ S \to S + S \mid S * S \mid a \mid b \mid c$$

$$w\colon a + b\ * c$$

The string w can be derived from the context free grammar G in two different ways:

$$S \Rightarrow S + S \Rightarrow a + S \Rightarrow a + S * S \Rightarrow a + b * S \Rightarrow a + b * c$$

$$S \Rightarrow S * S \Rightarrow S + S * S \Rightarrow a + S * S \Rightarrow a + b * S \Rightarrow a + b * c$$

Note that the left-most variable is expanded at each step.

Definition: A left-most derivation is a derivation in which the left-most variable is replaced at each step.

Definition: A grammar G ambiguously derives w if there exists two left-most derivations of w. We say that G is ambiguous if this is true for some w.

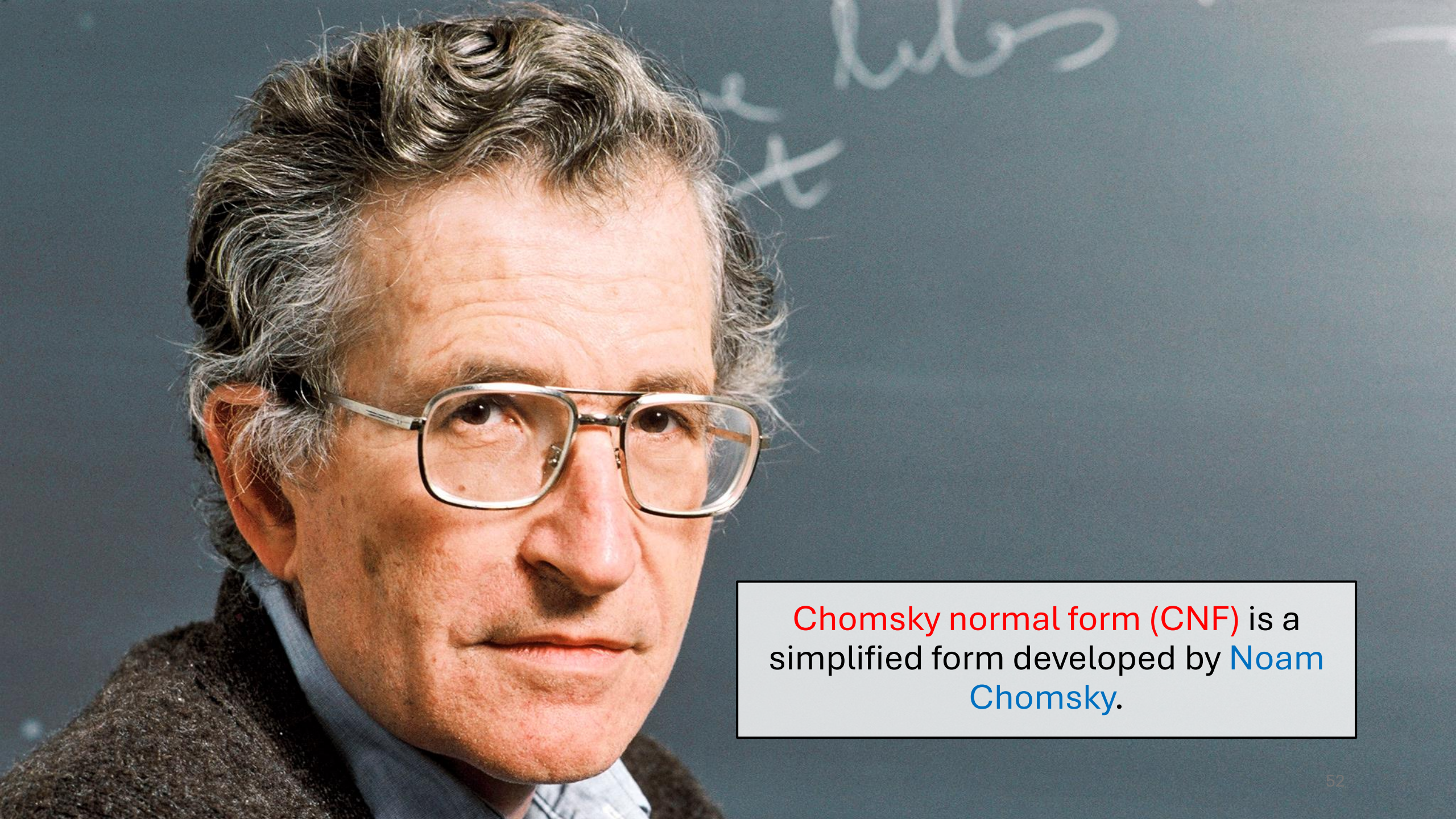
Sometimes the ambiguity that exists is inherent to the language and so every grammar for it is ambiguous.

$$L_{AMB} = \{0^i 1^j 2^k | i = j \text{ or } j = k\}$$

The Theory of Computing

Chomsky Normal Form

For computing and consistency, it is helpful to have grammars in a common form, usually called a normal form.



Chomsky normal form (CNF) is a simplified form developed by Noam Chomsky.

Definition: A context free grammar is in Chomsky normal form if every rule is of the form:

$$A \to BC$$
$$A \to \sigma$$

where σ is any terminal and A, B and C are variables. B and C may not be the start variable. In addition, the rule $S \to \epsilon$ is permitted only when S is the start variable.

Any context free grammar can be converted to Chomsky normal form.

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \epsilon$$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$S \rightarrow ASA \mid aB$
$A \rightarrow B \mid S$
$B \rightarrow b \mid \epsilon$

$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB$
$A \rightarrow B \mid S$
$B \rightarrow b \mid \epsilon$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$S_0 \rightarrow S$

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \epsilon$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid aB \mid a$

$A \rightarrow B \mid S \mid \epsilon$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid aB \mid a$

$A \rightarrow B \mid S \mid \epsilon$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid S \mid aB \mid a$

$A \rightarrow B \mid S$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid S \mid aB \mid a$

$A \rightarrow B \mid S$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow B \mid S$

$B \rightarrow b$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the new rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow B \mid S$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow S \mid b$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow S \mid b$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow ASA \mid AS \mid SA \mid aB \mid a \mid b$

$B \rightarrow b$

$S_0 \rightarrow S$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow ASA \mid AS \mid SA \mid aB \mid a \mid b$

$B \rightarrow b$

$S_0 \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$

$A \rightarrow ASA \mid AS \mid SA \mid aB \mid a \mid b$

$B \rightarrow b$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the new rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$S_0 \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$A \rightarrow ASA \mid AS \mid SA \mid aB \mid a \mid b$
$B \rightarrow b$

$S_0 \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a$
$S \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a$
$A \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a \mid b$
$A_a \rightarrow a$
$B \rightarrow b$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the new rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$S_0 \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a$

$S \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a$

$A \rightarrow ASA \mid AS \mid SA \mid A_aB \mid a \mid b$

$A_a \rightarrow a$

$B \rightarrow b$

$S_0 \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a$

$S_A \rightarrow SA$

$S \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a$

$A \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a \mid b$

$A_a \rightarrow a$

$B \rightarrow b$

To convert a grammar to Chomsky normal form we will perform five steps:

1. We create a new start variable S_0 and add the rule $S_0 \to S$ where S was the old start variable.
2. We eliminate ϵ rules of the type $A \to \epsilon$. If $B \to uAv$ is a rule, where u and v are strings of variables and terminals, we add the new rule $B \to uv$ (replacing A with ϵ).
3. We eliminate unit rules of the type $A \to B$. If $B \to u$ is a rule, where u is a string of variables and terminals, we add the rule $A \to u$.
4. For each terminal σ create the rule $B_\sigma \to \sigma$. Replace all occurrences of σ on the right-hand side of rules with B_σ.
5. For each rule $A \to u_1 u_2 \cdots u_k$ where $k > 2$ replace this rule with $A \to u_1 B$ and $B \to u_2 \cdots u_k$. Recursively apply this step until there are no rules of this form.

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \epsilon$$

$$S_0 \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a$$
$$S_A \rightarrow SA$$
$$S \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a$$
$$A \rightarrow AS_A \mid AS \mid SA \mid A_aB \mid a \mid b$$
$$A_a \rightarrow a$$
$$B \rightarrow b$$

Exercises

Can we convert these grammars to CNF?

$$S \to A \mid B$$
$$A \to 0A1 \mid \epsilon$$
$$B \to 1B0 \mid \epsilon$$

$$S \to A0011A$$
$$A \to 0A \mid 1A \mid \epsilon$$

$$S \to 0A0 \mid 1B1$$
$$A \to 0A \mid \epsilon$$
$$B \to 1B \mid \epsilon$$